

```
In [5]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

data = pd.read_csv('iris.csv')
print(data.head())

X = data.iloc[:, :-1]
y = data.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

def plot_confusion_matrix(cm, model_name):
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
    plt.title(f'Confusion Matrix for {model_name}')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()

print("\n--- Gaussian Naïve Bayes ---")
gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred_gnb = gnb.predict(X_test)
print(f"Confusion Matrix:\n{confusion_matrix(y_test, y_pred_gnb)}")
print(f"Accuracy: {accuracy_score(y_test, y_pred_gnb) * 100:.2f}%")
print(f"Precision: {precision_score(y_test, y_pred_gnb, average='weighted', zero_division=0):.2f}")
print(f"Recall: {recall_score(y_test, y_pred_gnb, average='weighted', zero_division=0):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred_gnb, average='weighted', zero_division=0):.2f}")
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	4.39	4.38	1.18	0.53	Iris-setosa
1	4.41	2.69	1.80	0.56	Iris-setosa
2	4.59	2.32	1.20	0.42	Iris-setosa
3	5.56	3.71	1.45	0.29	Iris-setosa
4	5.60	3.03	1.38	0.37	Iris-setosa

--- Gaussian Naïve Bayes ---

Confusion Matrix:

```
[[72  0  0]
```

```
 [ 0 56  5]
```

```
 [ 0 14 54]]
```

Accuracy: 90.55%

Precision: 0.91

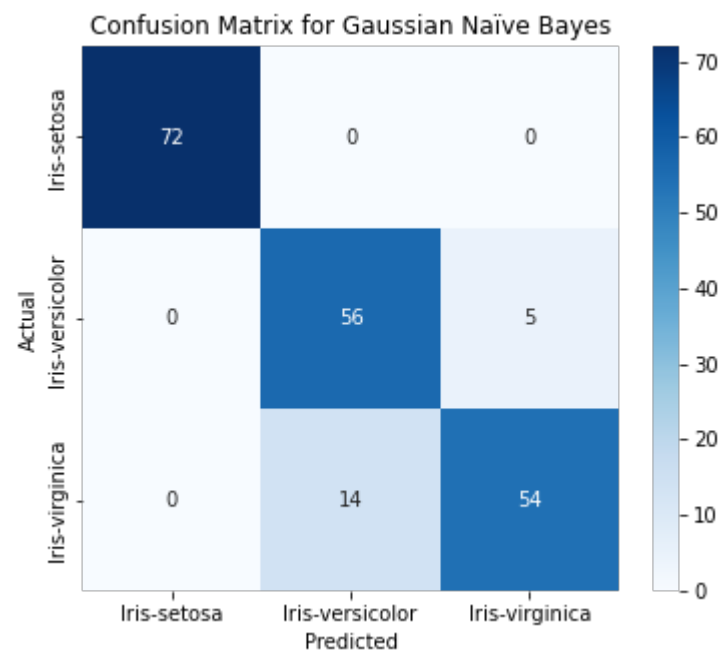
Recall: 0.91

F1 Score: 0.91

```
In [2]: # Confusion Matrix in Gaussian Bayes
cm_gnb = confusion_matrix(y_test, y_pred_gnb)
print(f"Confusion Matrix:\n{cm_gnb}")
plot_confusion_matrix(cm_gnb, 'Gaussian Naïve Bayes')
```

Confusion Matrix:

```
[[72  0  0]
 [ 0 56  5]
 [ 0 14 54]]
```



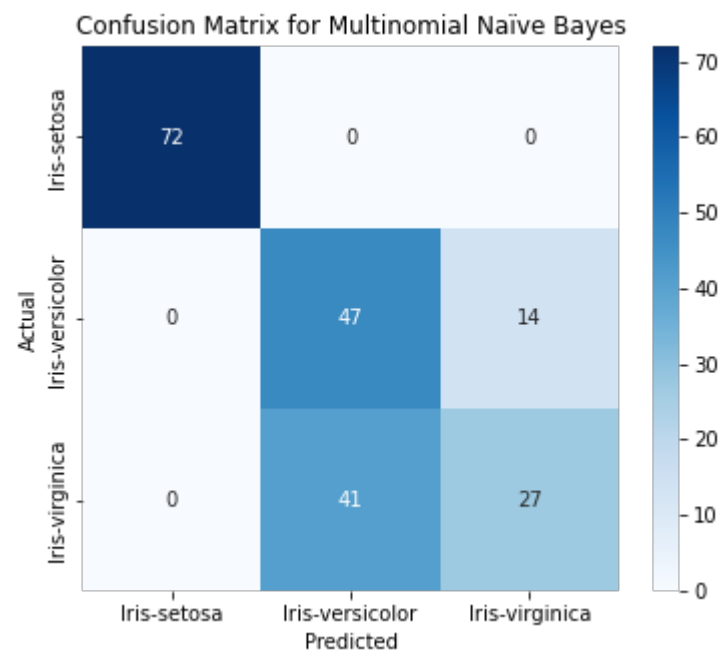
```
In [3]: print("\n--- Multinomial Naïve Bayes ---")
mnb = MultinomialNB()
mnb.fit(X_train, y_train)
y_pred_mnb = mnb.predict(X_test)
print(f"Confusion Matrix:\n{confusion_matrix(y_test, y_pred_mnb)}")
print(f"Accuracy: {accuracy_score(y_test, y_pred_mnb) * 100:.2f}%")
print(f"Precision: {precision_score(y_test, y_pred_mnb, average='weighted', zero_division=0):.2f}")
print(f"Recall: {recall_score(y_test, y_pred_mnb, average='weighted', zero_division=0):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred_mnb, average='weighted', zero_division=0):.2f}")
```

```
--- Multinomial Naïve Bayes ---
Confusion Matrix:
[[72  0  0]
 [ 0 47 14]
 [ 0 41 27]]
Accuracy: 72.64%
Precision: 0.74
Recall: 0.73
F1 Score: 0.72
```

```
In [4]: # Confusion Matrix in Multinomial Naive Bayes
cm_mnb = confusion_matrix(y_test, y_pred_mnb)
print(f"Confusion Matrix:\n{cm_mnb}")
plot_confusion_matrix(cm_mnb, 'Multinomial Naïve Bayes')
```

Confusion Matrix:

```
[[72  0  0]
 [ 0 47 14]
 [ 0 41 27]]
```



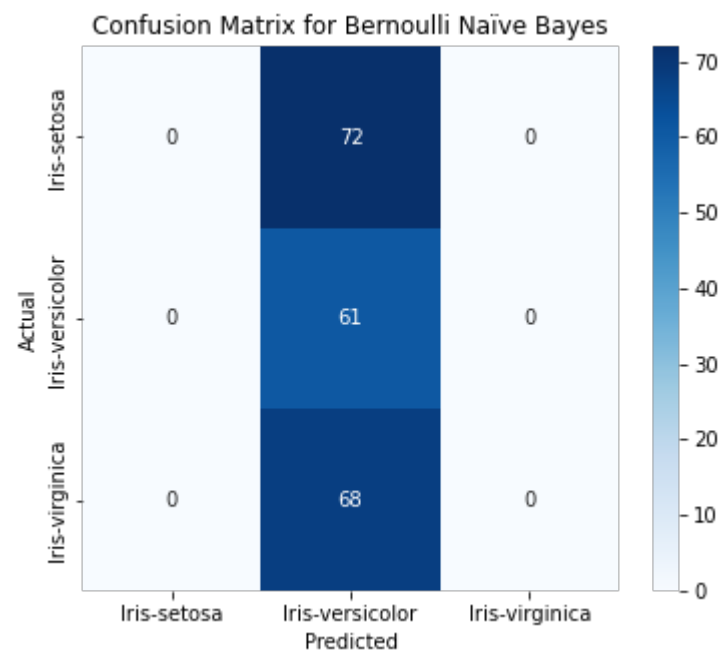
```
In [5]: print("\n--- Bernoulli Naïve Bayes ---")
        bnb = BernoulliNB()
        bnb.fit(X_train, y_train)
        y_pred_bnb = bnb.predict(X_test)
        print(f"Confusion Matrix:\n{confusion_matrix(y_test, y_pred_bnb)}")
        print(f"Accuracy: {accuracy_score(y_test, y_pred_bnb) * 100:.2f}%")
        print(f"Precision: {precision_score(y_test, y_pred_bnb, average='weighted', zero_division=0):.2f}")
        print(f"Recall: {recall_score(y_test, y_pred_bnb, average='weighted', zero_division=0):.2f}")
        print(f"F1 Score: {f1_score(y_test, y_pred_bnb, average='weighted', zero_division=0):.2f}")
```

```
--- Bernoulli Naïve Bayes ---
Confusion Matrix:
[[ 0 72  0]
 [ 0 61  0]
 [ 0 68  0]]
Accuracy: 30.35%
Precision: 0.09
Recall: 0.30
F1 Score: 0.14
```

```
In [6]: # Confusion Matrix in Bernoulli Naïve Bayes
cm_bnb = confusion_matrix(y_test, y_pred_bnb)
print(f"Confusion Matrix:\n{cm_bnb}")
plot_confusion_matrix(cm_bnb, 'Bernoulli Naïve Bayes')
```

Confusion Matrix:

```
[[ 0 72  0]
 [ 0 61  0]
 [ 0 68  0]]
```



In []: